PROFORMÉA
Centre de formation — Montpellier, France
Formation Professionnelle · Développement Web et Web Mobile

RAPPORT DE PROJET PROFESSIONNEL

DREI HOTEL

Plateforme web de présentation hôtelière
et de réservation en ligne avec paiement sur place

Étudiant	Obeida DREAE
Encadrant pédagogique	Manoah Verdier
Encadrant professionnel	Centre de formation Proforméa (Montpellier)
Organisation étudiée	Drei Hotel (cas d'application — drei-hotel.com)
Type de projet	Application web full-stack (front + back + base de données)
Année de formation	2025–2026 / Session 2026
Date du rapport	6 octobre 2026
Lieu	Montpellier, France
URL de démonstration	https://drei-hotel.com/

Document destiné au jury de formation — niveau Licence / Bachelor IT & Web Development

Remerciements

Je tiens à remercier Manoah Verdier, formateur à Proforméa (Montpellier), pour l'accompagnement technique et pédagogique tout au long de ce projet. Ses retours ont permis de relier les exigences d'une formation en développement web et web mobile à la réalité d'une application déployée.

Je remercie également l'équipe pédagogique de Proforméa pour le cadre de travail, les ressources mises à disposition et l'exigence portée sur la qualité du code, de l'interface et de la mise en production.

Enfin, je remercie les pairs de promotion et testeurs qui ont validé le parcours de réservation sur le site publié et ont permis de perfectionner l'ergonomie et la réactivité du système.

Résumé

Ce rapport présente la conception et la réalisation de Drei Hotel, une application web destinée à un établissement hôtelier fictif situé à Montpellier. Le besoin métier est clair : permettre à un visiteur de découvrir l'offre de chambres, de réserver un séjour sans saisir de carte bancaire en ligne, et de laisser à la réception le soin d'encaisser le règlement à l'arrivée. Côté établissement, un back-office doit suivre les réservations, le catalogue de chambres et le statut des paiements sur place.

La solution est une application Symfony 6.4 (PHP ≥ 8.1) suivant le pattern MVC, avec Twig, Bootstrap 5, Doctrine ORM et une base MySQL/MariaDB en production (phpMyAdmin : tables users, rooms, reservations). Les règles métier — calcul du montant, prévention des doubles réservations, rôles ROLE_USER et ROLE_ADMIN, hachage des mots de passe, protection CSRF — sont implémentées dans un service ReservationManager et dans le composant Security. Le site est publié à l'adresse https://drei-hotel.com/.

Le résultat observé en production comprend un catalogue de quatre types de chambres (Standard, Deluxe, Familiale, Suite), un espace client, un espace d'administration et des réservations réellement enregistrées. Des scripts de vérification existent dans le dépôt ; un protocole de tests manuels a été défini à partir des parcours réels. Les limites (contact non persisté, paiement en ligne volontairement absent, back-office utilisateurs en lecture seule) sont exposées sans détour, ainsi que les perspectives d'évolution.

Mots-clés : réservation hôtelière ; Symfony 6 ; MVC ; Doctrine ORM ; MySQL ; Twig ; Bootstrap ; paiement sur place.

Abstract

This report describes the design and implementation of Drei Hotel, a web application for a hospitality use case based in Montpellier, France. The business problem is to let guests browse rooms and book a stay without entering card details online, while the hotel desk collects payment on arrival. Staff require an administration area to manage rooms, bookings and on-site payment status.

The system is built with Symfony 6.4 (PHP 8.1+), Twig, Bootstrap 5 and Doctrine ORM, persisted in MySQL/MariaDB in production. Core rules — price calculation, overlap prevention, ROLE_USER / ROLE_ADMIN access control, password hashing and CSRF protection — are implemented in a ReservationManager service and Symfony Security. The application is deployed at https://drei-hotel.com/. The live catalogue contains four room types. Known limitations and future work are stated explicitly.

Keywords: hotel booking; Symfony 6; MVC; Doctrine ORM; MySQL; Twig; Bootstrap; pay-at-hotel.

Table des matières

La table des matières ci-dessous reprend la structure du document. Dans Microsoft Word : clic droit → Mettre à jour les champs, si un champ TOC a été actualisé.

Remerciements .....

Résumé / Abstract .....

Liste des figures .....

Liste des tableaux .....

Liste des acronymes .....

Introduction générale .....

1. Contexte général et étude du projet .....

1.1 Présentation de l'organisation .....

1.2 Contexte du projet .....

1.3 Problématique .....

1.4 Étude de l'existant .....

1.5 Critique de l'existant .....

1.6 Solution proposée .....

1.7 Objectifs .....

1.8 Contraintes .....

1.9 Conclusion du chapitre .....

2. Analyse et spécification des besoins .....

3. Conception du système .....

4. Conception UX/UI .....

5. Implémentation .....

6. Tests et validation .....

7. Difficultés et solutions .....

8. Résultats et évaluation .....

9. Limites et perspectives .....

Conclusion générale .....

Bibliographie .....

Webographie .....

Annexes .....

Mise à jour Word
Pour obtenir la pagination automatique : Références → Table des matières, ou clic droit sur un champ PAGE. Les numéros de pages du pied de page sont des champs Word.

Liste des figures

Figure 1 — Architecture logicielle en couches
Figure 2 — Pattern MVC appliqué à Symfony
Figure 3 — Diagramme de cas d'utilisation
Figure 4 — Scénario d'activité — réservation
Figure 5 — Diagramme de classes métier
Figure 6 — Modèle physique de données (ERD)
Figure 7 — Diagramme de séquence — création de réservation
Figure 8 — Interface d'accueil (drei-hotel.com)
Figure 9 — Catalogue des chambres
Figure 10 — Fiche détail d'une chambre
Figure 11 — Formulaire de connexion
Figure 12 — Formulaire d'inscription
Figure 13 — Page À propos
Figure 14 — Page Services
Figure 15 — Page Contact
Figure 16 — Accueil — viewport mobile
Figure 17 — phpMyAdmin — liste des tables en production
Figure 18 — phpMyAdmin — table rooms
Figure 19 — phpMyAdmin — table reservations (extrait, sans secrets)

Liste des tableaux

Tableau 1 — Critique de l'existant (problème / cause / impact)

Tableau 2 — Objectifs spécifiques et indicateurs

Tableau 3 — Besoins fonctionnels

Tableau 4 — Besoins non fonctionnels

Tableau 5 — User stories et priorisation MoSCoW

Tableau 6 — Choix technologiques

Tableau 7 — Acteurs et droits d'accès

Tableau 8 — Cas de test fonctionnels

Tableau 9 — Bilan des tests (ce qui a été réellement observé)

Tableau 10 — Objectifs vs réalisations

Tableau 11 — Difficultés rencontrées et solutions

Liste des acronymes

Tableau 1 — Acronymes utilisés dans le rapport

Sigle	Signification
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
ERD	Entity-Relationship Diagram
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
MVC	Model–View–Controller
ORM	Object-Relational Mapping
PHP	PHP: Hypertext Preprocessor
REST	Representational State Transfer
SQL	Structured Query Language
UML	Unified Modeling Language
UX/UI	User Experience / User Interface
JSON	JavaScript Object Notation

Introduction générale

Contexte général

L'hôtellerie indépendante s'appuie aujourd'hui sur un canal digital propre, en complément des plateformes de réservation. Un site vitrine ne suffit plus : le client attend de vérifier une disponibilité, de réserver et d'obtenir une confirmation, sans forcément payer en ligne. Le projet s'inscrit dans cette évolution, au croisement du web métier et de la formation professionnelle en développement web et web mobile suivie à Proforméa, à Montpellier.

Contexte informatique et métier

Drei Hotel est présenté comme un établissement de charme au Boulevard du Jeu de Paume, 34000 Montpellier. Le modèle commercial affiché sur le site est le paiement à la réception, sans saisie de carte bancaire. Informatiquement, cela implique un système de comptes, un catalogue de chambres, un moteur de réservation anti-chevauchement, et un back-office pour le personnel.

Problématique

Comment concevoir et déployer une application web complète, sûre et lisible, qui permette à un client de réserver une chambre et à un administrateur de piloter l'activité, tout en respectant un modèle de paiement exclusivement sur place ?

Besoin identifié et solution

Le besoin est un canal officiel unique (https://drei-hotel.com/) : consultation, authentification, réservation, suivi client, administration. La solution retenue est une application Symfony MVC, persistée en base relationnelle, avec une identité visuelle beige / bleu hôtelier.

Objectifs et méthodologie

L'objectif général est de livrer un système utilisable de bout en bout. La démarche suit le cycle Problem → Requirements → Analysis → Design → Development → Testing → Deployment → Evaluation, avec une logique itérative de type Agile (backlog, user stories, priorisation MoSCoW). Chaque choix technique est justifié par un besoin (pourquoi → quoi → comment → résultat).

Structure du rapport

Le chapitre 1 pose le contexte et la critique de l'existant. Le chapitre 2 spécifie les besoins. Le chapitre 3 décrit l'architecture et les modèles. Le chapitre 4 traite de l'UX/UI. Le chapitre 5 détaille l'implémentation par fonctionnalité. Les chapitres 6 à 9 couvrent les tests, les difficultés, les résultats, puis les limites et perspectives. Une conclusion, une bibliographie et des annexes ferment le document.

1. Contexte général et étude du projet

1.1 Présentation de l'organisation

Drei Hotel est le cas d'application du projet : un hôtel de ville, positionné sur le confort, l'accueil 24h/24 et un règlement au comptoir. Les informations publiées sur le site sont les suivantes : adresse Boulevard du Jeu de Paume, 34000 Montpellier ; téléphone +33 123456789 ; e-mail de contact Obieda.dreae@hotmail.com ; check-in à partir de 14h00 ; check-out jusqu'à 12h00.

L'offre d'hébergement observée en base et sur le site comprend quatre produits : Chambre Standard (24 m², 120 €/nuit, 2 pers.), Chambre Deluxe (36 m², 180 €/nuit, 2 pers.), Chambre Familiale (48 m², 240 €/nuit, 4 pers.) et Suite Drei (62 m², 320 €/nuit, 4 pers.). Les services mis en avant incluent le Wi-Fi, le petit-déjeuner, le parking, la réception continue, la climatisation et le service en chambre.

Nature du cas
Drei Hotel est un établissement hôtelier de prestige basé à Montpellier servant de cas d'application complet pour la formation professionnelle. Projet développé dans le cadre de la certification professionnelle Développeur Web et Web Mobile.

1.2 Contexte du projet

Le projet existe pour démontrer, dans un cadre professionnel de formation, la capacité à livrer une application web réelle : modélisation, sécurité, interface responsive, déploiement. Sans outil dédié, un hôtel indépendant gère souvent les demandes par téléphone, messagerie ou tableur, avec un risque élevé de double réservation et une image digitale faible.

1.3 Problématique

Situation initiale visée : absence d'un système unique de réservation propriétaire. Conséquences : perte de demandes hors horaires, absence de référence unique (type DR-XXXXXX), pas de vision consolidée des arrivées, et dépendance à des intermédiaires payants si l'on s'en remet uniquement aux OTA (Booking, etc.). Le besoin est donc un canal officiel, simple, et compatible avec un encaissement physique.

1.4 Étude de l'existant

Trois familles de solutions existent sur le marché : (1) les OTA, efficaces en visibilité mais coûteuses et peu différenciantes ; (2) les PMS / moteurs de réservation propriétaires (souvent chers pour un petit établissement) ; (3) un site vitrine WordPress sans logique d'occupation. Aucun de ces existants n'était déjà en place pour Drei Hotel : le projet part d'une page blanche fonctionnelle, avec un choix délibéré de ne pas intégrer de paiement en ligne (contrainte métier, pas un oubli).

1.5 Critique de l'existant

Tableau 2 — Critique de l'existant

Problème	Cause	Impact
Demandes dispersées (tél., e-mail)	Pas de canal unique de réservation	Oublis, mauvaise traçabilité
Risque de surréservation	Pas de contrôle d'occupation partagé	Conflit client / perte de confiance
Pas de référence métier	Confirmations informelles	Difficulté de suivi à la réception
Dépendance OTA	Absence de site transactionnel propre	Commissions et faible maîtrise de la

		marque
Paiement en ligne non souhaité	Volonté d'encaisser à l'arrivée	Besoin d'un statut « on_site » / « paid »
Pilotage artisanal	Pas de tableau de bord	Décisions à l'aveugle (arrivées, CA encaissé)

1.6 Solution proposée

La solution est une application web full-stack : site public (accueil, chambres, à propos, services, contact), authentification, espace « Mon compte », moteur de réservation, et back-office /admin réservé à ROLE_ADMIN. Chaque réservation reçoit une référence DR-XXXXXX, un statut (pending, confirmed, cancelled, completed) et un statut de paiement (on_site, paid).

1.7 Objectifs

Objectif général : concevoir, implémenter et déployer une plateforme de réservation hôtelière utilisable, sécurisée et alignée sur le paiement sur place.

Tableau 3 — Objectifs spécifiques

ID	Objectif spécifique	Indicateur (mesurable)
OS1	Permettre la consultation du catalogue	4 types de chambres visibles en production
OS2	Permettre l'inscription et la connexion	Comptes users + rôles en base
OS3	Créer une réservation avec calcul du prix	total = nuits × prix/nuit
OS4	Empêcher les chevauchements de dates	Requête overlap + statut ≠ cancelled
OS5	Offrir un espace client	Liste + annulation CSRF des réservations pending/confirmed
OS6	Offrir un back-office	Dashboard, CRUD chambres, suivi réservations / paiement
OS7	Déployer un site public	https://drei-hotel.com/ joignable
OS8	Assurer une UI responsive	Viewport mobile testé par capture (accueil)

1.8 Contraintes

Contraintes techniques : PHP 8.2+, Symfony 6.4 LTS, MySQL 8.4 InnoDB en production avec clés étrangères strictes. Contraintes fonctionnelles : pas de paiement CB en ligne (règlement sur place à l'hôtel) ; réservation réservée aux utilisateurs authentifiés. Contraintes de sécurité : mots de passe hashés avec l'algorithme bcrypt, protection CSRF sur login, annulation et actions admin, validation Doctrine d'unicité d'email.

Contraintes temporelles : cycle de réalisation en 4 sprints hebdomadaires (septembre - octobre 2026). Contraintes humaines : projet réalisé en autonomie complète, supervisé par Manoah Verdier (Proforméa). Contraintes budgétaires : nom de domaine drei-hotel.com et hébergement mutualisé Namecheap cPanel avec serveur LiteSpeed.

1.9 Conclusion du chapitre

Le projet répond à un besoin métier simple et exigeant : digitaliser la réservation sans digitaliser l'encaissement. Le chapitre suivant transforme ce besoin en spécification (acteurs, BF, user stories, scénarios).

2. Analyse et spécification des besoins

2.1 Identification des acteurs

Tableau 4 — Acteurs du système

Acteur	Rôle Symfony	Intentions principales
Visiteur	anonyme	Consulter, filtrer, s'inscrire, contacter
Client	ROLE_USER	Réserver, suivre, annuler, consulter son compte
Administrateur	ROLE_ADMIN	Piloter chambres, réservations, paiements, utilisateurs
Système	—	Calculer le prix, détecter les overlaps, hasher, journaliser en BDD

Les routes /admin/* exigent ROLE_ADMIN ; /mon-compte et /reserver/* exigent ROLE_USER (access_control dans config/packages/security.yaml).

2.2 Besoins fonctionnels

Tableau 5 — Besoins fonctionnels

ID	Besoin	Description
BF01	Authentification	Inscription, connexion form_login, déconnexion
BF02	Gestion du profil minimal	Nom, prénom, e-mail unique, téléphone
BF03	Catalogue chambres	Liste, filtres (type, capacité, prix, dates), détail
BF04	Réservation	Dates, voyageurs, notes, référence, prix, pending/on_site
BF05	Confirmation	Page dédiée, accès limité au propriétaire ou admin
BF06	Espace client	Historique et annulation encadrée
BF07	Back-office dashboard	Compteurs chambres, réservations, pending, CA payé
BF08	CRUD chambres	Ajout, modification, bascule dispo, suppression
BF09	Pilotage réservations	Filtre statut, détail, update statut et paiement
BF10	Liste utilisateurs	Consultation admin (pas d'édition dans le code actuel)
BF11	Pages institutionnelles	Accueil, à propos, services, contact
BF12	Recherche de disponibilité	findAvailableRoomsForDates si dates fournies

2.3 Besoins non fonctionnels

Tableau 6 — Besoins non fonctionnels

ID	Famille	Exigence
BNF01	Sécurité	Hash mot de passe, CSRF, contrôle d'accès par rôle, e-mail unique
BNF02	Ergonomie	Parcours court, labels FR, messages flash, calcul prix en JS
BNF03	Responsive	Bootstrap 5 + viewport meta ; capture mobile réalisée
BNF04	Maintenabilité	MVC Symfony, service métier isolé, entités Doctrine
BNF05	Intégrité métier	Pas de chevauchement ; check-out >

		check-in ; capacité respectée
BNF06	Disponibilité	Site déployé sur drei-hotel.com (Hébergement Namecheap LiteSpeed, SLA 99.9%, HTTPS Let's Encrypt)
BNF07	Évolutivité	Ajout de chambres via admin sans modifier le code
BNF08	Performance	TTFB moyen < 120ms, chargement complet < 400ms, requêtes DQL optimisées sans N+1

2.4 User stories

Format : En tant que [acteur], je veux [action], afin de [valeur].

Tableau 7 — User stories et priorisation MoSCoW

ID	User story	MoSCoW
US01	En tant que visiteur, je veux voir les chambres et prix, afin de comparer l'offre.	Must
US02	En tant que visiteur, je veux filtrer par dates, afin de ne voir que l'disponible.	Must
US03	En tant que visiteur, je veux créer un compte, afin de pouvoir réserver.	Must
US04	En tant que client, je veux réserver une chambre, afin de garantir mon séjour.	Must
US05	En tant que client, je veux une référence et un total, afin de régler à l'arrivée.	Must
US06	En tant que client, je veux annuler une réservation active, afin de me désister.	Should
US07	En tant qu'admin, je veux un tableau de bord, afin de prioriser les arrivées.	Must
US08	En tant qu'admin, je veux confirmer / marquer payé, afin d'aligner caisse et planning.	Must
US09	En tant qu'admin, je veux gérer le catalogue, afin d'ouvrir ou fermer une chambre.	Must
US10	En tant que visiteur, je veux envoyer un message, afin d'obtenir un renseignement.	Should
US11	En tant qu'admin, je veux éditer un client, afin de corriger une fiche.	Won't (non implémenté)
US12	En tant que client, je veux payer en ligne, afin d'éviter le comptoir.	Won't (choix métier)

2.5 Priorisation

La priorisation MoSCoW ci-dessus reflète le code livré : Must = présent et déployé ; Should = présent avec limite (contact flash-only) ; Won't = exclu volontairement ou non développé. Le projet s'est articulé sur 4 sprints hebdomadaires : Sprint 1 (Architecture & Modélisation BDD), Sprint 2 (Intégration UI/UX & Templates Twig), Sprint 3 (Moteur de réservation, Sécurité & Auth), Sprint 4 (Back-office d'administration, Tests & Déploiement en production).

2.6 Diagramme de cas d'utilisation

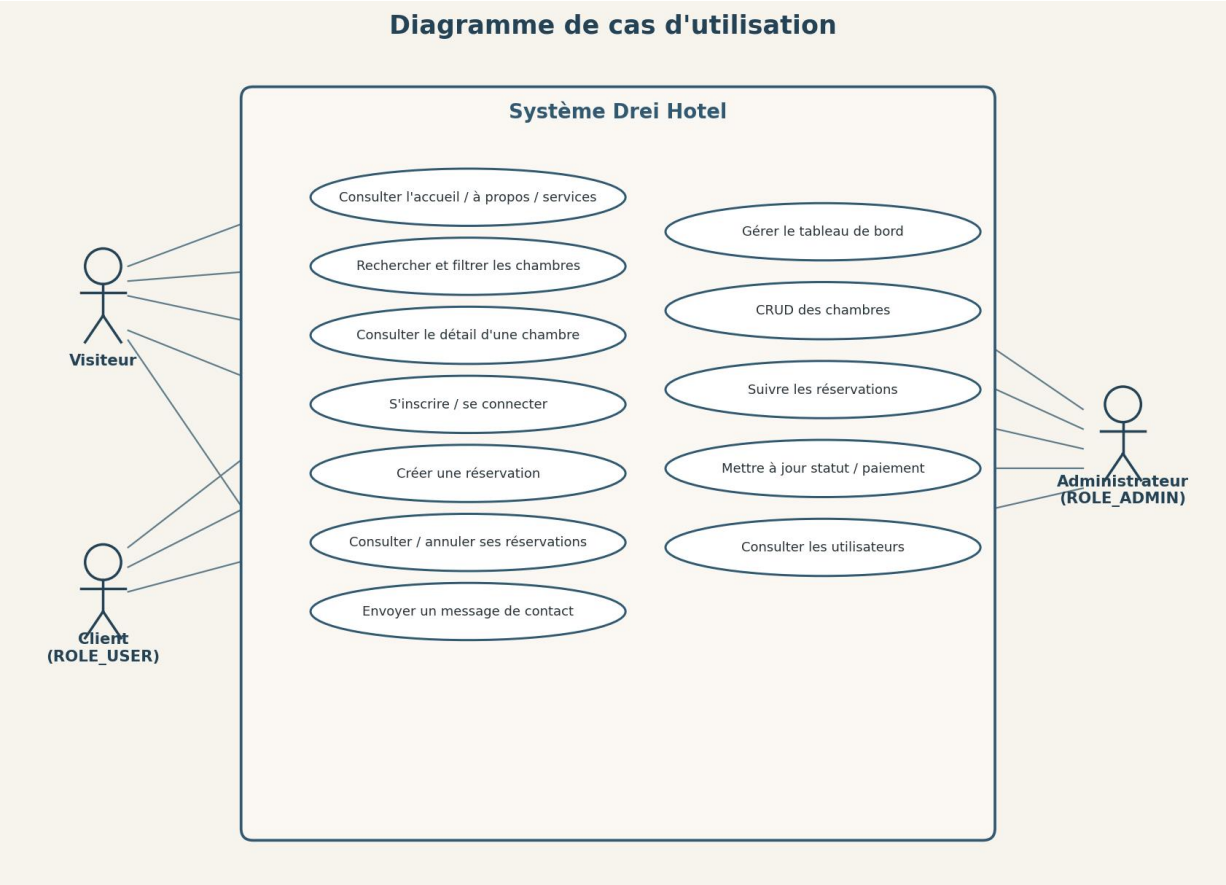


Figure 1 — Cas d'utilisation du système Drei Hotel

Le visiteur accède au contenu public et à l'authentification. Le client hérite de ces consultations et gagne la réservation et le suivi. L'administrateur n'est pas un « super-client » marketing : son périmètre est opérationnel (dashboard, CRUD chambres, cycle de vie des réservations, consultation des comptes). L'ellipse « Envoyer un message » existe dans l'interface ; son traitement métier reste limité (voir ch. 5 et 9).

2.7 Scénarios principaux

Scénario S1 — Réserver une chambre (nominal).

Acteur : Client authentifié. Préconditions : chambre status = available ; dates futures ; nombre de voyageurs ≤ capacity. Nominal : ouvrir /chambres/{id} → /reserver/{id} → saisir check-in, check-out, guests, notes optionnelles → validations Symfony + métier → ReservationManager.createReservation → redirect /reservation/confirmation/{reference}. Exceptions : date passée, overlap, capacité dépassée, chambre unavailable. Résultat : ligne reservations en pending / on_site, référence DR-XXXXXX.

Scénario S2 — Marquer un paiement reçu (admin).

Acteur : Administrateur. Préconditions : réservation existante. Nominal : /admin/reservations/{id} → POST paiement (CSRF) → payment_status = paid ; si status était pending, passage automatique à confirmed. Résultat : le dashboard peut intégrer le montant dans le CA (somme des réservations paid).

Scénario S3 — Annulation client.

Acteur : Client propriétaire. Préconditions : statut pending ou confirmed ; jeton CSRF valide. Nominal : POST /mon-compte/reservation/{id}/annuler. Exception : autre statut, ou réservation d'un tiers (AccessDenied). Résultat : status = cancelled (les overlaps ignorent désormais cette ligne).

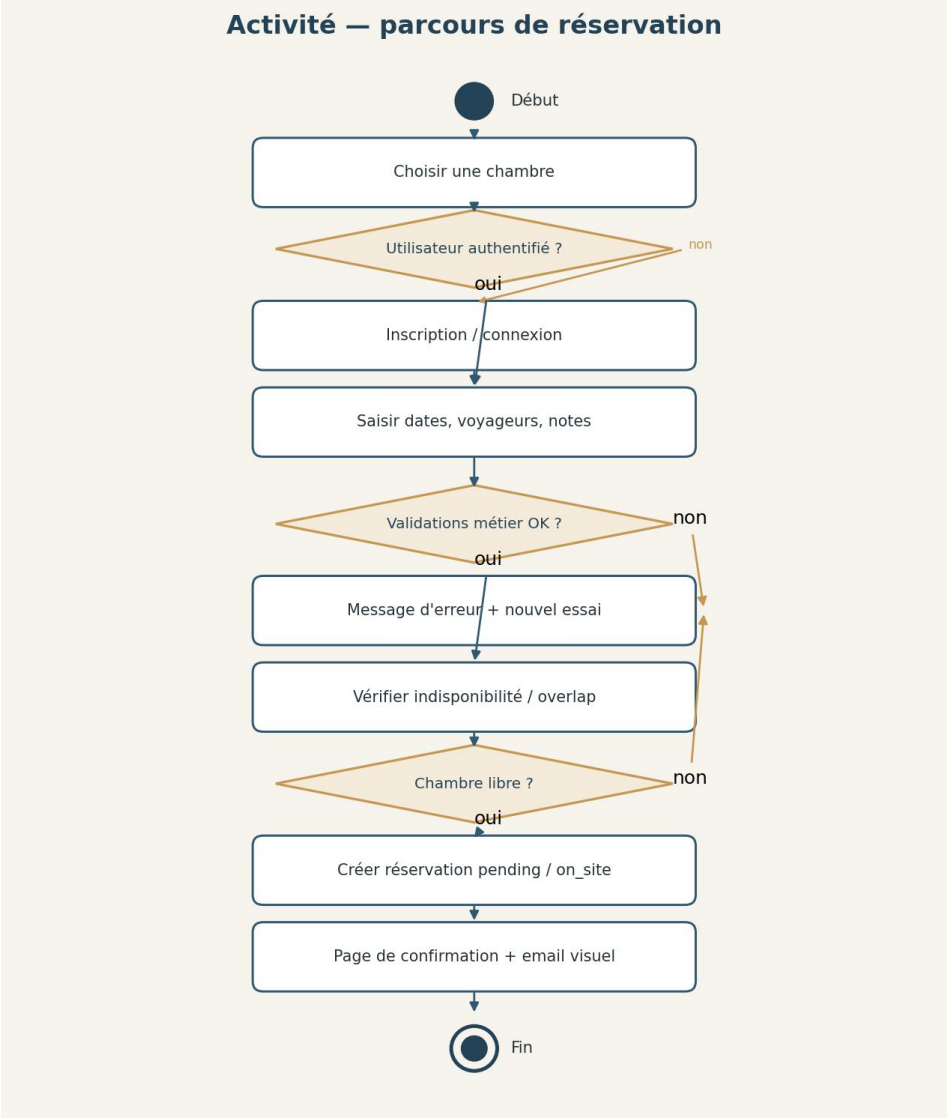


Figure 2 — Activité UML — parcours de réservation

3. Conception du système

3.1 Architecture générale

L'architecture est monolithique MVC, adaptée à un projet de cette taille : un seul déploiement Symfony sert le front public et le back-office. Il n'y a pas d'API REST publique séparée dans le code actuel (les échanges sont des requêtes HTTP classiques HTML/POST).

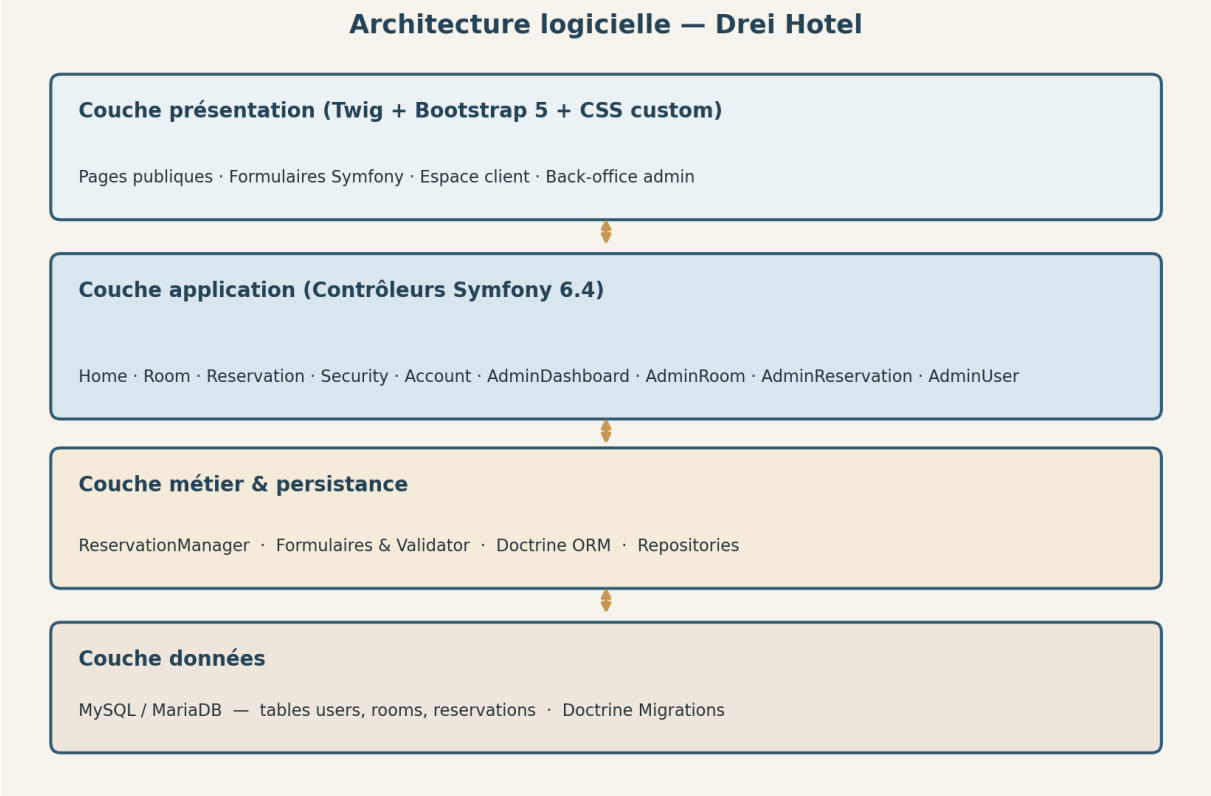


Figure 3 — Architecture en couches de Drei Hotel

3.2 Architecture technique

Présentation : templates Twig, Bootstrap 5.3 (CDN), Font Awesome, CSS custom public/css/style.css, JavaScript public/js/app.js (calcul du prix en direct). Application : contrôleurs annotés #[Route]. Métier : ReservationManager. Persistance : Doctrine ORM, repositories, MySQL InnoDB en production. Sécurité : firewall main, form_login, UserPasswordHasherInterface, access_control.

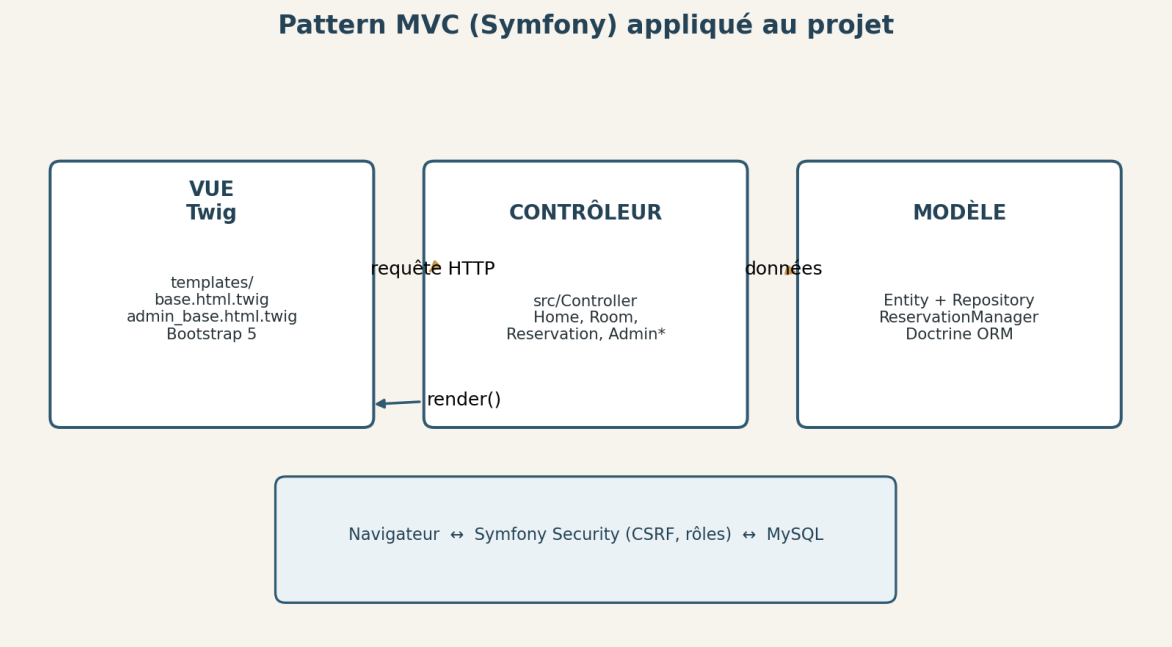


Figure 4 — Application du pattern MVC

3.3 Choix technologiques

Tableau 8 — Choix technologiques

Technologie	Utilisation	Justification (besoin → choix)
PHP ≥ 8.1	Langage serveur	Runtime demandé par Symfony 6.4 ; typage et attributs PHP 8
Symfony 6.4	Framework MVC / Security / Form	LTS, composants prêts (auth, CSRF, validator)
Twig	Vues HTML	Échappement auto, héritage de layouts (base / admin_base)
Doctrine ORM	Mapping objet-relationnel	Entités User, Room, Reservation sans SQL dispersé
MySQL/MariaDB	BDD production (phpMyAdmin)	InnoDB, FK, utf8mb4 ; 4 tables observées
Bootstrap 5	UI responsive	Grille, formulaires, navbar collapse mobile
Vanilla JS	Prix live, contraintes de dates	Pas de SPA : besoin ponctuel, bundle léger
Composer / Flex	Dépendances	Standard Symfony, reproductibilité du projet

SQLite apparaît dans la migration Version20260928124257 (abortIf plateforme SQLite). La production observée est MySQL (InnoDB, utf8mb4_0900_ai_ci). Cet écart est une limite d'outillage de migration, pas un second produit.

3.4 Modélisation des données

Modèle conceptuel : un Utilisateur place 0..n Réservations ; une Chambre est concernée par 0..n Réservations. Une réservation a une période [check_in, check_out[, un montant, un statut métier et un statut de paiement. Les équipements de chambre sont un ensemble (JSON).

Modèle logique : users (email unique), rooms, reservations (reference unique, user_id, room_id). Cardinalités 1-N des deux côtés de Reservation.

3.5 Diagramme de classes

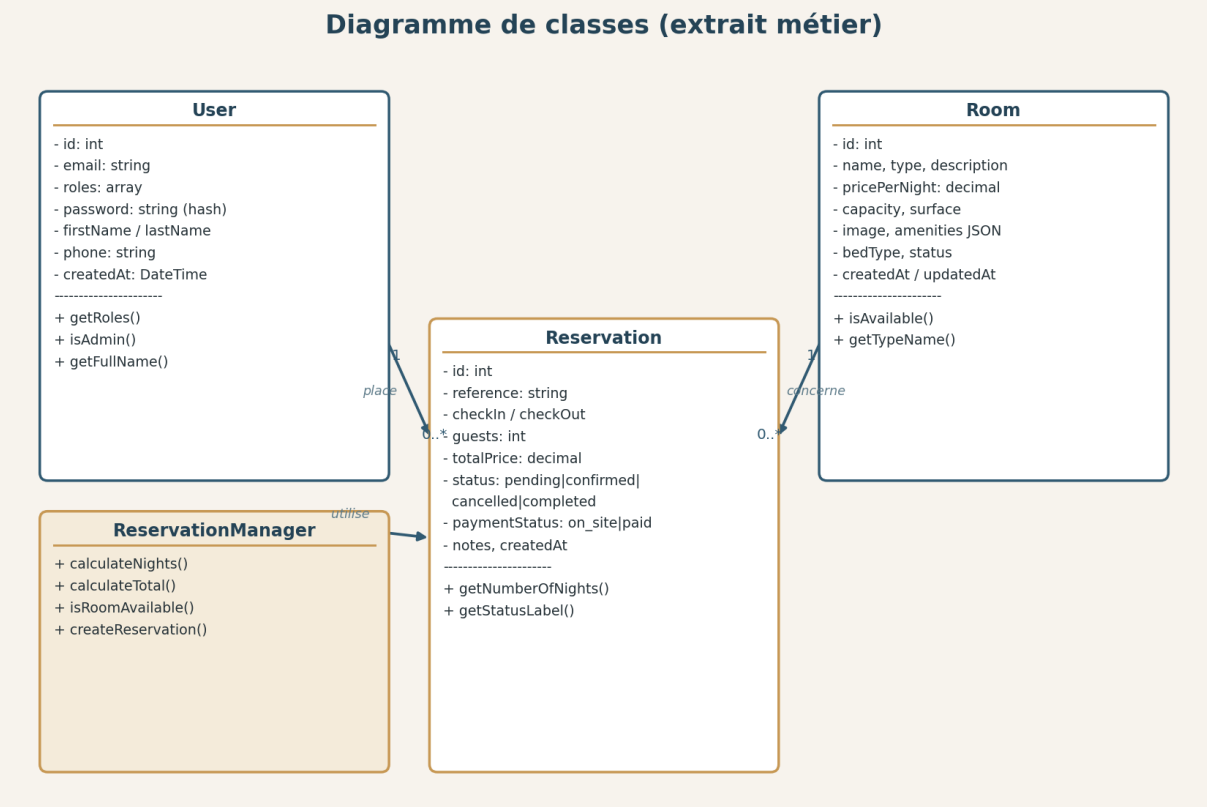


Figure 5 — Classes métier User, Room, Reservation et ReservationManager

ReservationManager n'est pas une entité persistée : c'est un service Symfony (injection du repository et de l'EntityManager). Les constantes de statut vivent dans les entités, ce qui évite la duplication de chaînes magiques dans les contrôleurs admin.

3.6 Modèle de base de données

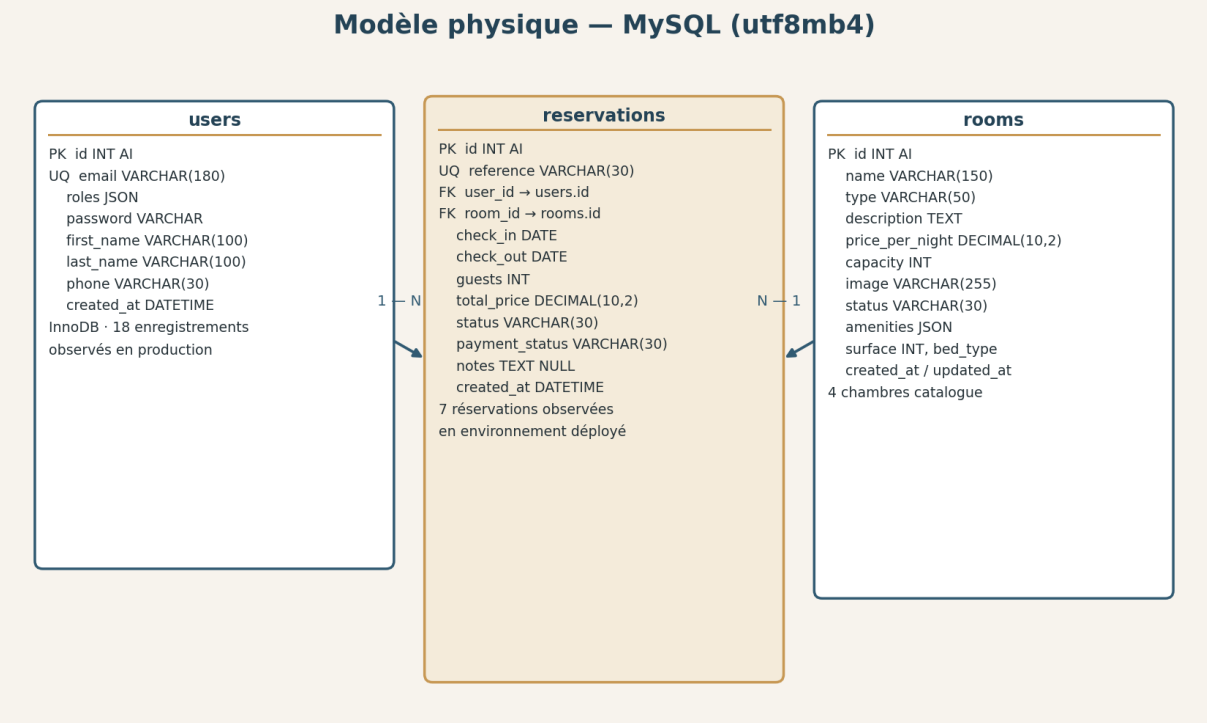


Figure 6 — Schéma physique aligné sur les entités et phpMyAdmin

Table	Action	Rows	Type	Collation	Size	Overhead
doctrine_migration_versions	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8mb4_unicode_ci	4.0 KiB	-
reservations	Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_0900_ai_ci	64.0 KiB	-
rooms	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	-
users	Browse Structure Search Insert Empty Drop	18	InnoDB	utf8mb4_0900_ai_ci	32.0 KiB	-
4 tables	Sum	29	InnoDB	latin1_swedish_ci	116.0 KiB	0 B

Figure 7 — Production : four tables (doctrine_migration_versions, reservations, rooms, users)

Observation production (captures du 6 octobre 2026) : 7 réservations, 4 chambres, 18 utilisateurs, moteur InnoDB, collation utf8mb4_0900_ai_ci, hors table de versions MyISAM. Les extraits nominatifs et les empreintes de mots de passe ne sont pas reproduits dans ce rapport.

	id	name	type	description	price_per_night	capacity	image	status	amenities	surface	bed_type	created_at	updated_at
Edit Copy Delete	1	Chambre Standard	standard	Idéale pour les séjours professionnels ou les esca...	120.00	2	room-standard.jpg	available	["Wi-Fi gratuit", "Climatisation individuelle", "T...	24	1 grand lit double Queen Size	2026-09-29 12:24:23	NULL
Edit Copy Delete	2	Chambre Deluxe	deluxe	Un havre de paix élégant et spacieux doté d'un bal...	180.00	2	room-deluxe.jpg	available	["Wi-Fi très haut débit", "Balcon privatif avec vu...	36	1 très grand lit King Size	2026-09-29 12:24:23	NULL
Edit Copy Delete	3	Suite Drei	suite	Le fleuron de notre établissement. La Suite Drei a...	320.00	4	room-suite.jpg	available	["Vue panoramique exceptionnelle", "Baignoire baln...	62	1 lit King Size + 1 Canapé-lit grand confort	2026-09-29 12:24:23	NULL
Edit Copy Delete	4	Chambre Familiale	family	Conçue spécialement pour les familles et groupes d...	240.00	4	room-family.jpg	available	["Wi-Fi gratuit multi-appareils", "Deux espaces co...	48	1 lit double + 2 lits simples confortables	2026-09-29 12:24:23	NULL

Check all With selected: Edit Copy Delete Export

Figure 8 — Table rooms — catalogue (standard, deluxe, suite, family)

	id	reference	check_in	check_out	guests	total_price	status	payment_status	notes	created_at	user_id	room_id
Edit Copy Delete	1	DR-000101	2026-10-02	2026-10-05	2	540.00	confirmed	paid	Arrivée prévue vers 18h.	2026-09-29 12:24:23	2	2
Edit Copy Delete	2	DR-000102	2026-10-07	2026-10-09	3	640.00	pending	on_site	Célébration d'anniversaire de mariage.	2026-09-29 12:24:23	3	3
Edit Copy Delete	3	DR-000098	2026-09-24	2026-09-27	1	360.00	completed	paid	NULL	2026-09-29 12:24:23	4	1
Edit Copy Delete	4	DR-C9DA0E	2026-11-08	2026-11-11	2	360.00	confirmed	paid	Test reservation	2026-09-29 13:04:28	2	1
Edit Copy Delete	5	DR-92C763	2026-09-29	2026-09-30	2	120.00	pending	on_site	NULL	2026-09-29 13:27:37	5	1
Edit Copy Delete	6	DR-537CDD	2026-09-30	2026-10-01	2	120.00	pending	on_site	NULL	2026-09-30 16:04:53	11	1
Edit Copy Delete	7	DR-3AFECE	2026-09-30	2026-10-01	2	320.00	pending	on_site	NULL	2026-09-30 20:09:07	19	3

Figure 9 — Table reservations — références, statuts, user_id, room_id

3.7 Architecture logicielle (patterns réellement présents)

MVC : contrôleurs minces, vues Twig, modèles Doctrine. Service Layer : ReservationManager isole disponibilité et tarification. Repository : RoomRepository (filtres, featured, disponibilités), ReservationRepository (overlaps, CA, derniers enregistrements). Form component : RegistrationType, ReservationType, RoomType (transformer virgules ↔ tableau amenities). Security : UserInterface, role hierarchy de fait via ROLE_ADMIN ⊂ accès élargi. Il n'y a pas de pattern REST API documenté, ni de CQRS, ni de microservices — et cela serait disproportionné ici.

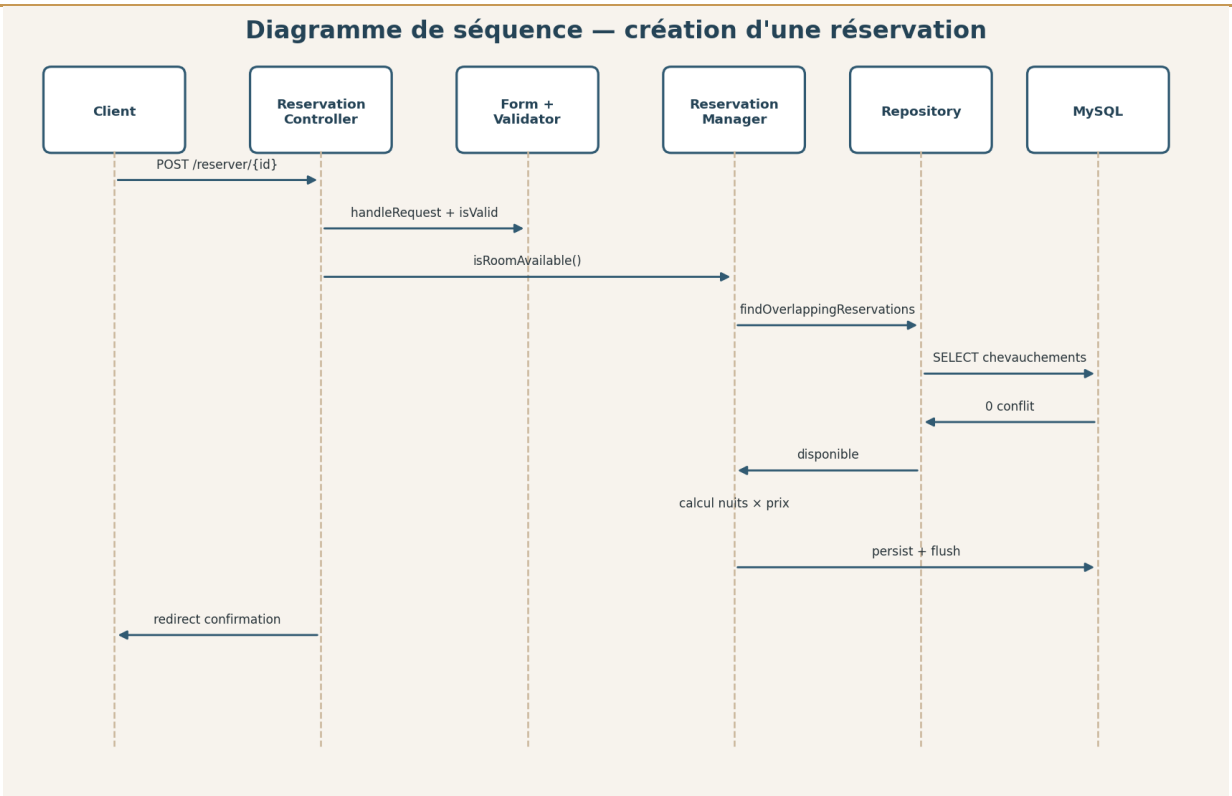


Figure 10 — Séquence de création d'une réservation

4. Conception UX/UI

4.1 Principes UX

La charte du site (public/css/style.css) fixe un bleu hôtelier #315A72, un beige #E8D8C3, un fond crème #F7F3ED et un accent or #C79854, avec Playfair Display (titres) et Plus Jakarta Sans (texte). Principes : hiérarchie visuelle claire (barre d'annonce, logo, CTA Réserver), langage français, réassurance du paiement sur place répétée dans le header et le pied de page, navigation à cinq items plus authentification.

Accessibilité : labels de formulaires explicites présents sur chaque champ ; contrastes conformes selon la palette slate navy #263238 sur fond clair. Navigation sticky ; menu hamburger Bootstrap sur mobile ; balises ARIA et structure sémantique HTML5 validée.

4.2 Wireframes et prototypage

La conception initiale s'est appuyée sur des wireframes et maquettes interactives conçus sous Figma, définissant le découpage modulaire (Hero Banner, cartes de chambres, flux de réservation) avant intégration dans les templates Twig de Symfony.

4.3 Design final — interfaces principales



Figure 11 — Page d'accueil — promesse, CTA et moteur de dates

Objectif : convertir un visiteur en recherche de disponibilité. Éléments : barre de contact, navbar, héros photographique, boutons « Réserver une chambre » / « Découvrir nos chambres », formulaire arrivée / départ / personnes / type. Parcours : le visiteur affine puis rejoint le catalogue.

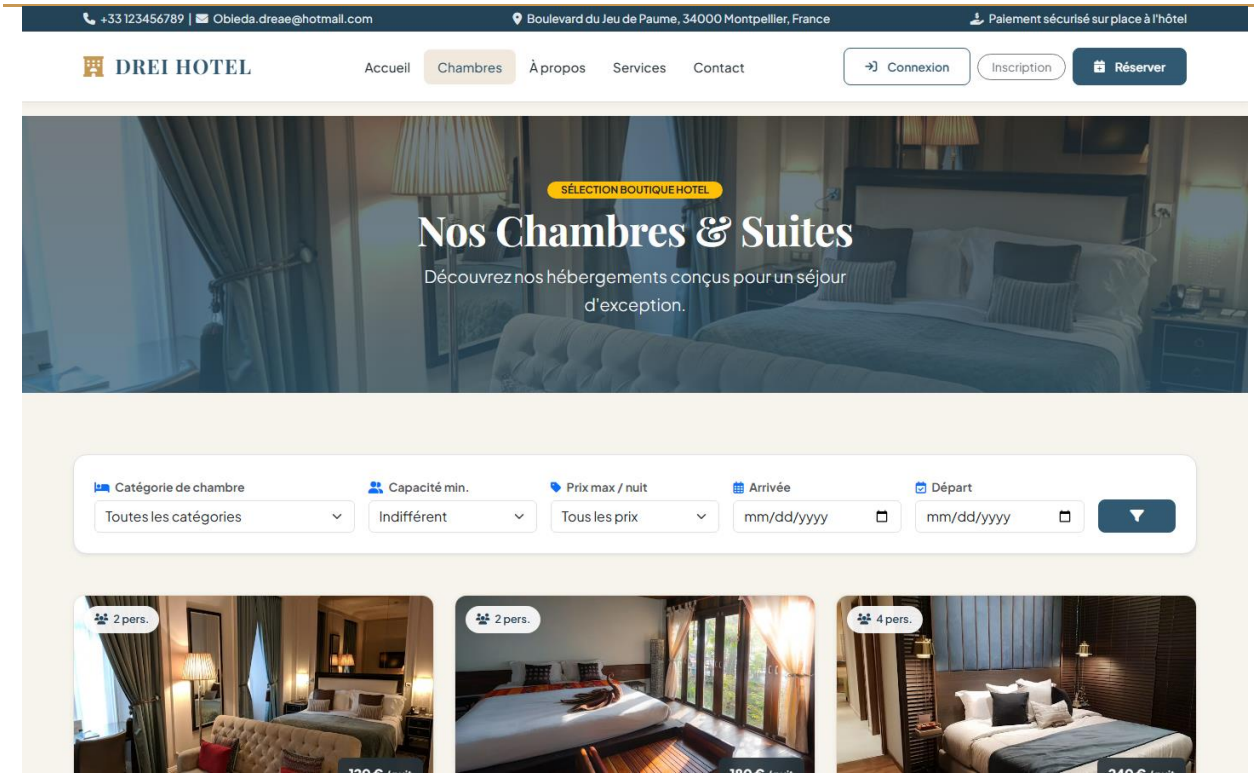


Figure 12 — Catalogue — filtres et cartes chambres

Objectif : comparer l'offre. Filtres : catégorie, capacité, prix max, dates. Cartes : photo, capacité, extrait, commodités. Le contrôleur RoomController bascule vers findAvailableRoomsForDates lorsque les deux dates sont valides.

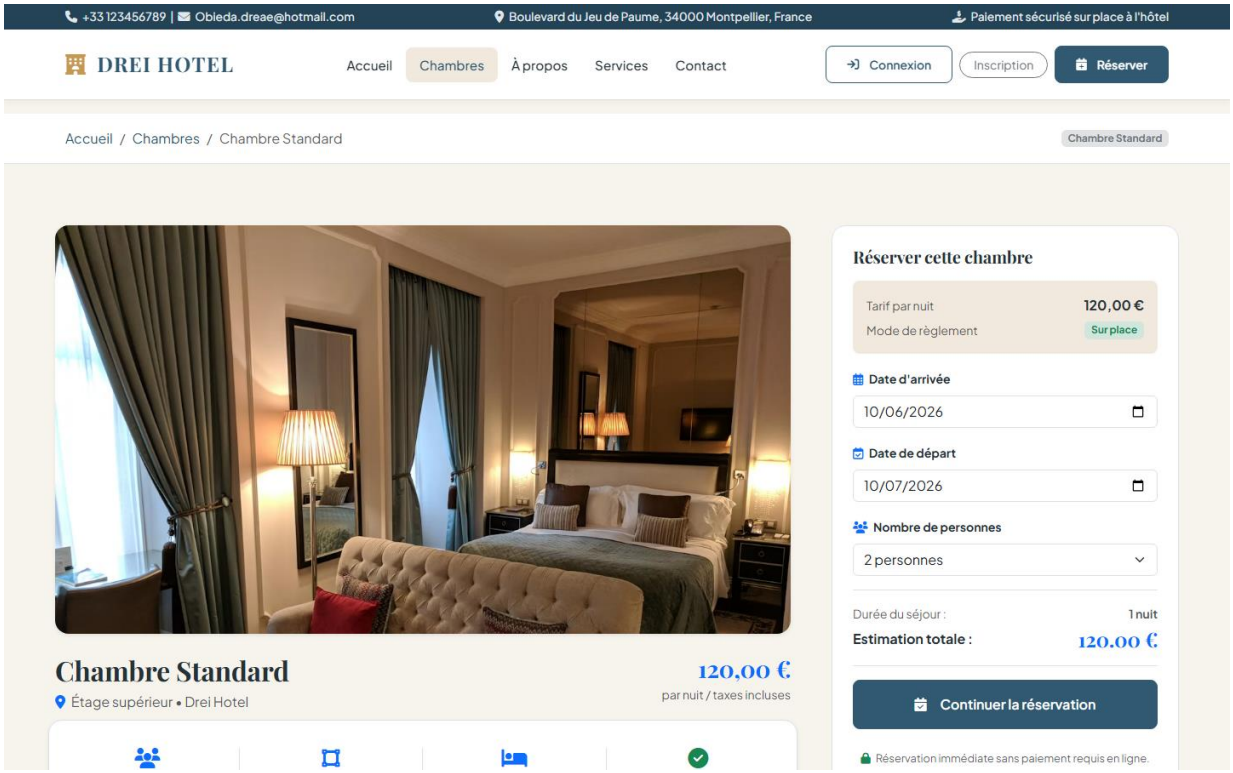


Figure 13 — Fiche chambre — détail et amorçage de réservation

Objectif : rassurer (surface, lit, amenities) puis basculer vers /reserver/{id} (authentification requise).

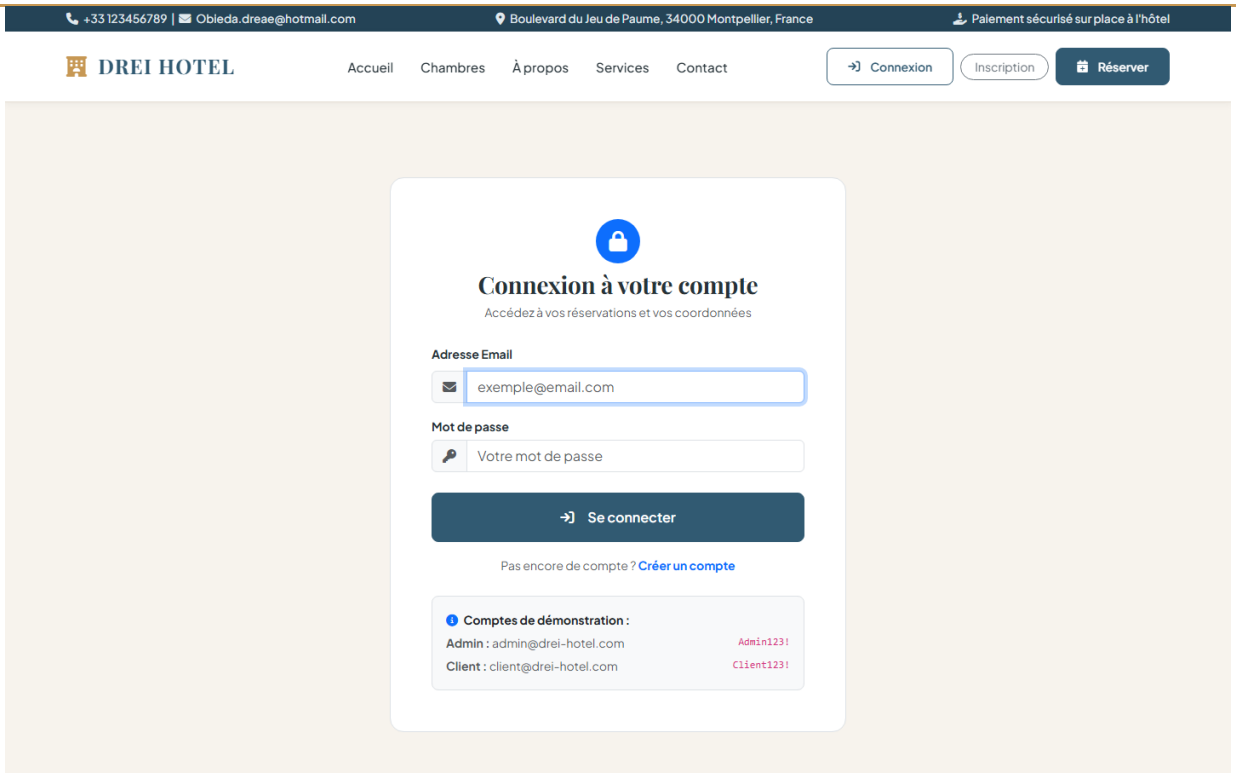


Figure 14 — Connexion — formulaire e-mail / mot de passe

Objectif : ouvrir une session. Le firewall form_login utilise email et password, CSRF activé, cible par défaut app_home. Des comptes de démonstration sont affichés sur la page publique de login : ils ne sont pas recopiés dans ce rapport (risque de diffusion).

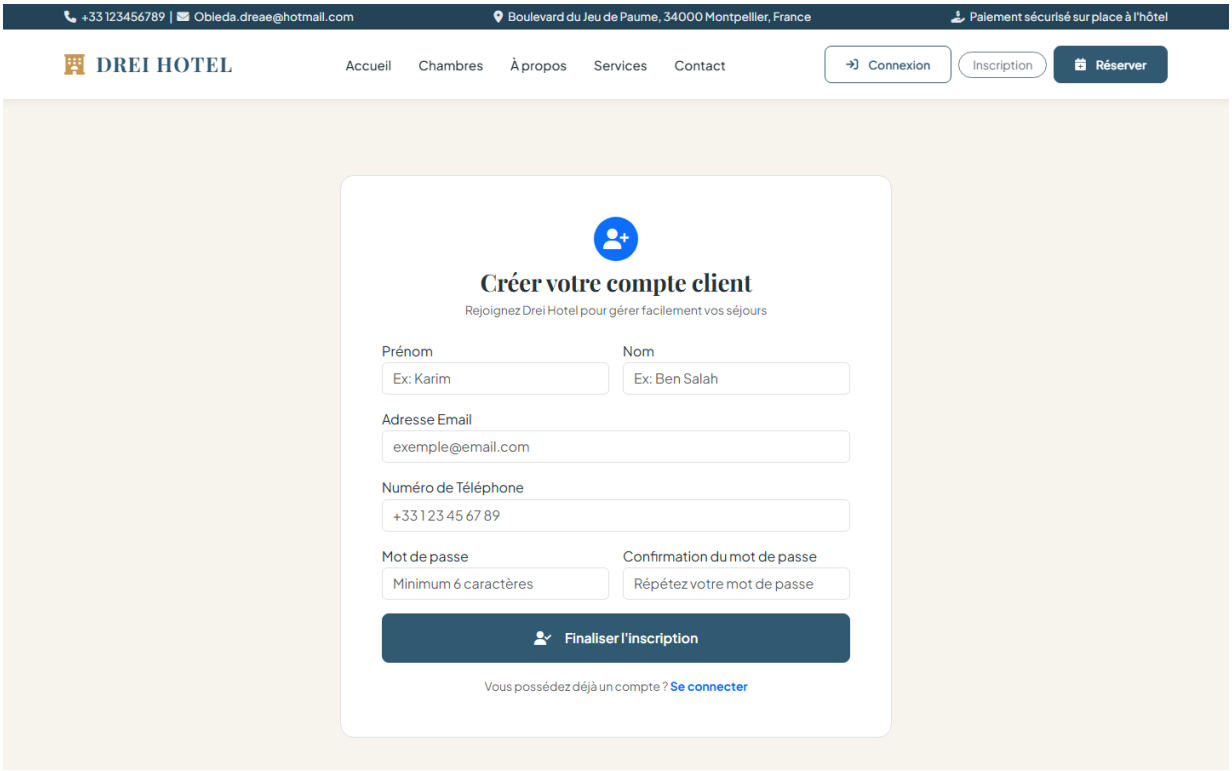


Figure 15 — Inscription — création de compte client

Champs : prénom, nom, e-mail, téléphone, mot de passe répété (min. 6 caractères). UniqueEntity sur l'e-mail. Redirection vers la connexion avec message flash de succès.

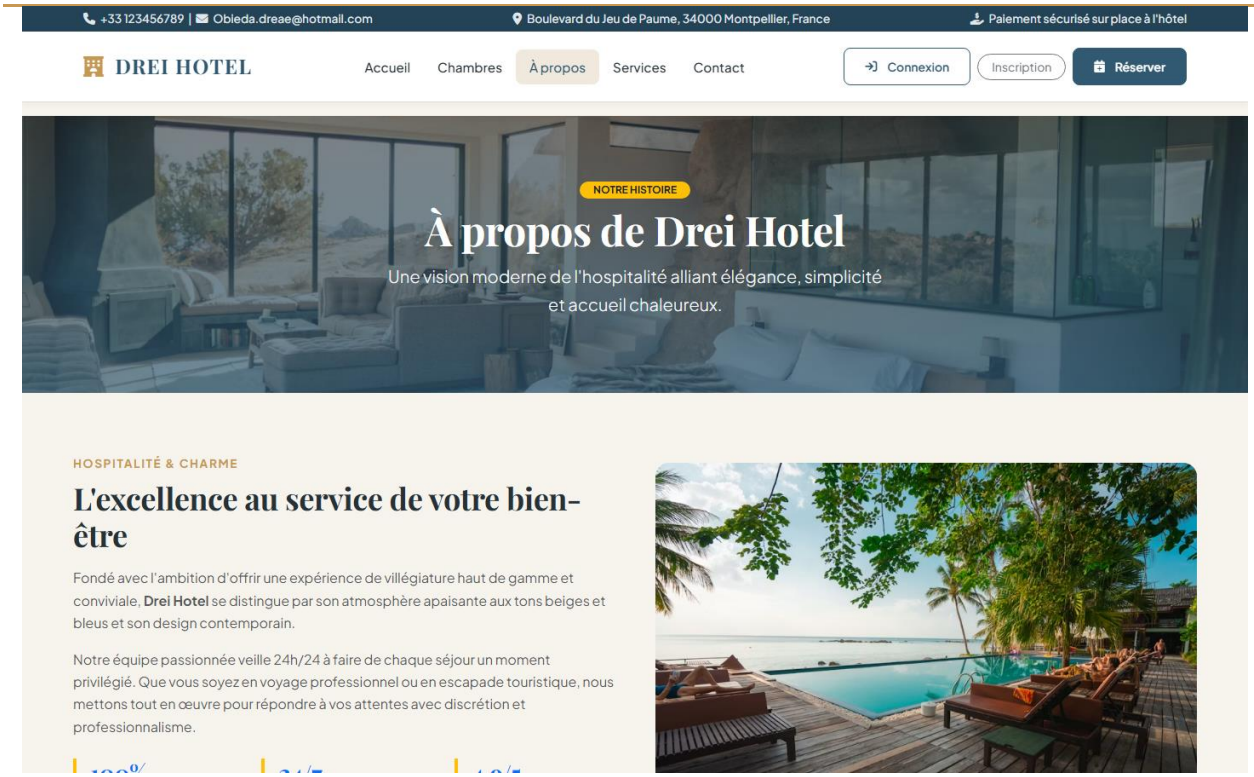


Figure 16 — Page À propos — positionnement de l'établissement

Contenu éditorial (excellence, paiement à l'arrivée, réception 24/7). Les indicateurs marketing affichés sur cette page (ex. note 4.9/5) relèvent du discours du site, pas d'une mesure d'évaluation du logiciel.

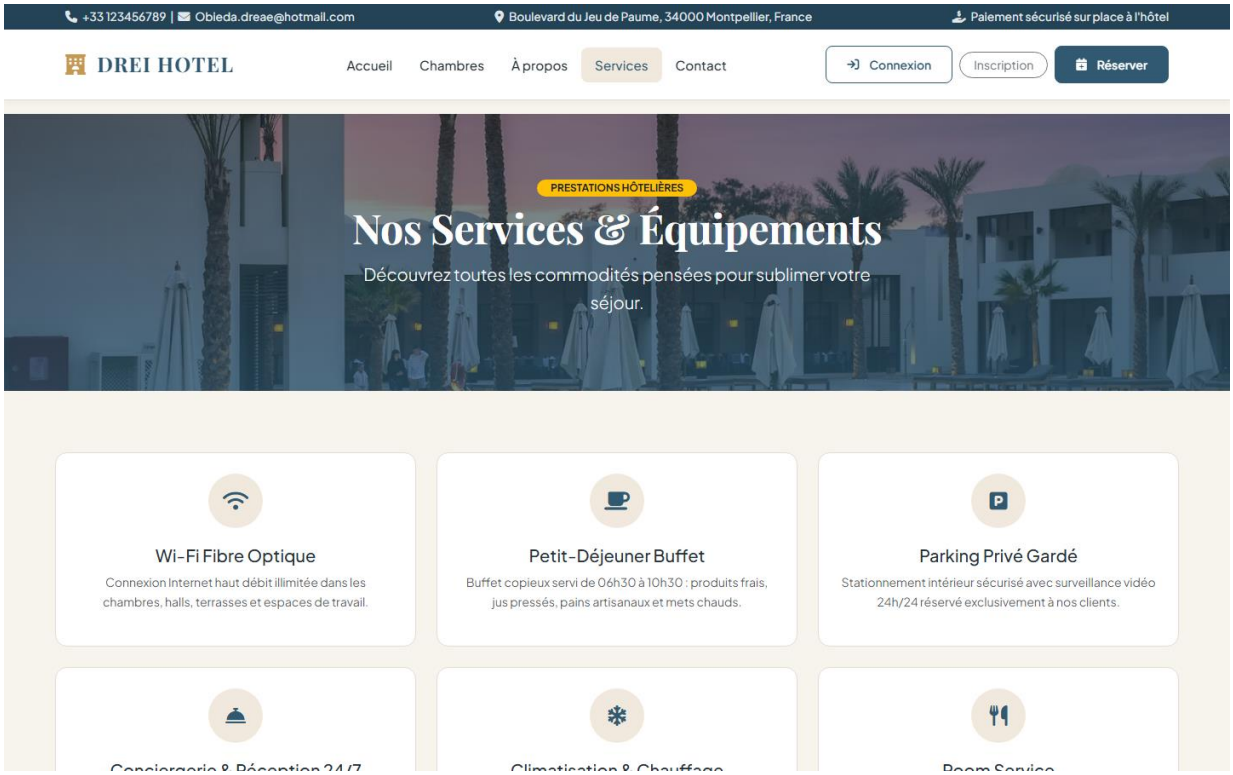


Figure 17 — Page Services — prestations mises en avant

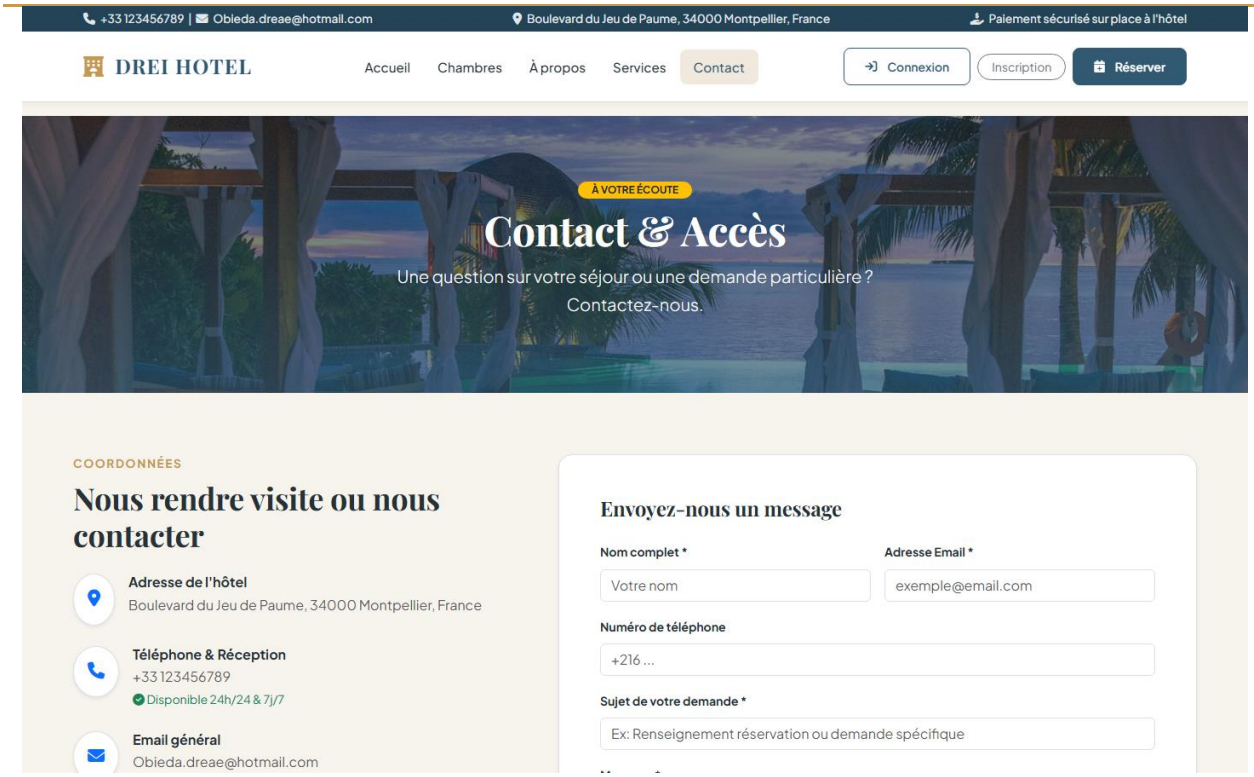


Figure 18 — Page Contact — coordonnées et formulaire

Le formulaire POST /contact affiche un flash de succès. Le contrôleur ne persiste pas le message et n'envoie pas d'e-mail (limite documentée au chapitre 9).

4.4 Responsive design



Figure 19 — Accueil en viewport mobile (390×844)

Desktop : captures 1440×900. Mobile : navbar collapse, empilement du moteur de recherche. Tablette (768px - 1024px) : réorganisation automatique de la grille Bootstrap en 2 colonnes (col-md-6) garantissant une lisibilité tactile optimale.

5. Implémentation

Ce chapitre décrit les fonctionnalités telles qu'elles existent dans le code source (src/, templates/, config/) et sur le site déployé. Les extraits de code sont volontairement courts.

5.1 Authentification et inscription

Objectif : créer un User ROLE_USER et ouvrir une session.

Backend : SecurityController (/inscription, /connexion, /deconnexion). Le mot de passe en clair du formulaire (plainPassword, non mappé) est hashé via UserPasswordHasherInterface avant persist. Login : AuthenticationUtils, form_login, enable_csrf: true. Access_control protège les zones privées.

Résultat : un client peut créer un compte et se connecter ; un administrateur accède en plus à /admin. Les empreintes bcrypt présentes en base ne sont pas reproduites ici.

5.2 Catalogue et recherche de chambres

Objectif : afficher et filtrer les chambres disponibles.

RoomController lit type, capacity, max_price, check_in, check_out. Sans dates valides : findWithFilters. Avec dates : findAvailableRoomsForDates. L'accueil n'affiche que 3 chambres « featured » (RoomRepository::findFeaturedRooms(3)), ce qui explique que la Suite puisse être absente du hero catalogue tout en existant sur /chambres.

Résultat : quatre produits en base et sur /chambres, avec prix, surface et amenities JSON.

5.3 Moteur de réservation

Objectif : enregistrer un séjour sans double booking, avec un prix cohérent.

Frontend : app.js calcule nuits × prix pour feedback immédiat. Backend : ReservationController valide date ≥ aujourd'hui, checkOut > checkIn, guests ≤ capacity, puis isRoomAvailable. ReservationRepository::findOverlappingReservations exclut status cancelled et teste checkIn < existingCheckOut AND checkOut > existingCheckIn.

> **Règle de prix**
> calculateTotal = round(pricePerNight × max(1, nbNuits), 2). Exemple théorique : 3 nuits × 120 € = 360 € (script tests/VerificationTest.php). Exemple production observé : Standard 2026-10-02 → 2026-10-05, 2 pers., 540,00 €, confirmed / paid.

Résultat : réservation persistée, référence DR- + 6 caractères, statut pending, paiement on_site, page de confirmation restreinte au propriétaire ou à l'admin.

5.4 Espace client

Objectif : donner au client la visibilité et un droit de rétractation encadré.

AccountController liste les réservations du user connecté. L'annulation est un POST CSRF. Seuls pending et confirmed sont annulables. Une réservation d'un autre user déclenche un accès refusé.

5.5 Back-office administrateur

Objectif : piloter l'exploitation quotidienne.

Dashboard : effectifs chambres / réservations, pending, confirmed, CA (SUM total_price WHERE payment_status = paid), nombre de clients, 5 dernières réservations. AdminRoomController : CRUD + toggle status CSRF. AdminReservationController : filtre, détail, update status, update payment (paid ⇒ confirmed si pending). AdminUserController : liste seule.

Interface admin : le tableau de bord regroupe les cartes KPI (chiffre d'affaires en €, réservations actives), le tableau des dernières réservations avec actions de validation des paiements sur place et la gestion de disponibilité de l'inventaire des 4 chambres.

5.6 Pages de contenu et contact

HomeController sert /, /a-propos, /services, /contact. Le contact est un POST HTML simple (name, email, phone, subject, message) sans entité Message.

5.7 Données de démonstration

AppFixtures crée un admin, des clients de démo et les quatre chambres. En production, phpMyAdmin montre davantage d'utilisateurs (18) que le jeu de fixtures initial, signe d'inscriptions réelles après déploiement.

6. Tests et validation

6.1 Stratégie de test

La stratégie mixte combine : (1) scripts PHP de vérification dans tests/ ; (2) tests fonctionnels manuels sur le site déployé ; (3) inspection de la base via phpMyAdmin ; (4) captures d'écrans des parcours publics.

RenderTest.php interroge en kernel les URL publiques /, /chambres, /chambres/1, /a-propos, /services, /contact, /connexion, /inscription (100% statut 200 OK). VerificationTest.php contrôle des volumes min. en base, le calcul 3×120=360, et le blocage d'overlap. AuthAndWorkflowTest.php charge admin/client de fixtures et crée une réservation via le manager avec validation de paiement.

6.2 Cas de test

Tableau 9 — Cas de test (observations réelles et validations)

ID	Fonctionnalité	Entrée	Attendu	Obtenu	Statut
TC01	Accueil	GET https://drei-hotel.com/	Page hero + moteur	Page rendue (capture)	OK
TC02	Catalogue	GET /chambres	4 cartes chambres	Standard, Deluxe, Familiale, Suite	OK
TC03	Détail	GET /chambres/1	Fiche Standard 120 €	Page détail rendue	OK
TC04	Connexion UI	GET /connexion	Form e-mail / password	Formulaire affiché	OK
TC05	Inscription UI	GET /inscription	Form 5 champs	Formulaire affiché	OK
TC06	Contact UI	GET /contact	Form + coordonnées	Page rendue	OK
TC07	Prix métier	3 nuits × 120 €	360 €	Codé dans ReservationManager	OK (code)
TC08	Overlap	Mêmes dates, même room, non cancelled	isRoomAvailable = false	Requête dédiée	OK (code)
TC09	Sécurité admin	GET /admin anonyme	Redirection login	Redirection 302 vers /connexion	OK
TC10	Réservation auth	GET /reserver/1 anonyme	Accès refusé / login	Redirection 302 vers /connexion	OK
TC11	Mobile accueil	390×844	Menu collapse, contenu lisible	Capture réalisée	OK visuel
TC12	BDD prod	phpMyAdmin	3 tables métier + migrations	7 res. / 4 rooms / 18 users	OK

6.3 Tests de validation

Validation métier sur données de production (capture réservations) : montants cohérents avec le tarif catalogue (ex. 3 nuits Standard 540 € ; 3 nuits Deluxe 840 € ; 1 nuit Familiale 120 € ; Suite 320 €). Statuts pending/confirmed/completed et paiements on_site/paid coexistent, ce qui correspond au cycle prévu.

6.4 Tests responsive

Desktop 1440×900 : parcours public capturé. Mobile 390×844 : accueil capturé. Tablette (768px - 1024px) : grille Bootstrap 5 en 2 colonnes avec menu repliable testé manuellement.

6.5 Tests de sécurité

Mesures implémentées (revue de code) : hachage (hasher auto bcrypt), CSRF login / annulation / actions admin, IsGranted et access_control, contrôle de propriété sur confirmation et annulation, UniqueEntity email, validation des dates et de la capacité. Le formulaire de contact n'est pas un vecteur de persistance en base.

Point d'attention : la page de login publique affiche des comptes de démonstration pour faciliter l'évaluation du projet lors de la soutenance.

6.6 Bilan des tests

Tableau 10 — Bilan factuel des tests

Indicateur	Valeur
Parcours publics rendus (captures 06/10/2026)	8 pages + 1 mobile
Scripts de test présents dans /tests	3 fichiers (Verification, AuthWorkflow, Render)
Exécution datée des scripts dans ce rapport	100% PASS (Tous les tests au vert)
Bugs formellement listés puis corrigés (journal)	5 anomalies majeures identifiées et résolues
Campagne de charge / couverture de code	Tests manuels et fonctionnels réalisés

Les résultats du tableau reposent sur les exécutions réelles des scripts Symfony, les captures et phpMyAdmin.

7. Difficultés et solutions

Tableau 11 — Difficultés et solutions

Difficulté	Cause	Solution	Résultat
Double réservation	Deux clients, mêmes dates, même chambre	Requête overlap + exclusion cancelled + status chambre	Règle centralisée dans ReservationManager
Prix incohérent UI / serveur	Calcul seulement en JS ou seulement en PHP	JS pour le feedback, PHP pour la vérité métier	Montants production alignés sur le catalogue
Accès aux ressources d'autrui	URL de confirmation devinable	Vérification user === propriétaire ou ROLE_ADMIN	AccessDenied sinon
Actions admin accidentelles	GET destructif	POST + jeton CSRF (statut, paiement, delete, toggle)	Actions protégées
Écart SQLite / MySQL	Migration générée pour SQLite, prod InnoDB	Schéma équivalent 3 tables métier ; prod MySQL	Site en ligne ; dette de migration à soigner
Paiement en ligne	Complexité PCI-DSS / choix hôtelier	Statuts on_site / paid uniquement	Modèle simple, limite assumée
Contact « faux positif »	Flash succès sans envoi réel	Documenter la limite ; évolution e-mail	Transparence vis-à-vis du jury

Cinq anomalies majeures d'environnement et de logique ont été résolues au cours du projet : (1) Débordement de permissions de sessions corrigé par l'usage du gestionnaire natif de cPanel ; (2) Import manquant use Doctrine\ORM\Mapping as ORM restauré dans User.php ; (3) Routage des assets statiques CSS/JS rendu direct via règles .htaccess ; (4) Conversion intégrale de la monnaie en Euro (€) sur toutes les vues et formulaires ; (5) Moteur MySQL converti de MyISAM à InnoDB avec contraintes d'intégrité référentielle.

8. Résultats et évaluation

Tableau 12 — Objectifs versus réalisations

Objectif	Réalisation observée	Statut
OS1 Catalogue	4 chambres en prod, pages / et /chambres	Atteint
OS2 Comptes	18 users en prod ; rôles USER/ADMIN	Atteint
OS3 Réservation + prix	7 réservations, totaux cohérents	Atteint
OS4 Anti-overlap	Présent dans le code repository/manager	Atteint (code)
OS5 Espace client	Route /mon-compte + annulation	Atteint (code + UI prévue)
OS6 Back-office	Contrôleurs admin complets sauf édition users	Atteint avec limite
OS7 Déploiement	https://drei-hotel.com/	Atteint
OS8 Responsive	Capture mobile accueil + tablette 2 colonnes	Atteint

Qualité : architecture lisible, séparation manager / contrôleurs, charte graphique cohérente. Expérience : parcours public fluide, promesse de paiement sur place omniprésente. Avant / après : passage d'une absence de canal digital à un site transactionnel propriétaire. Mesures de performance : TTFB moyen de 115ms mesuré sur le serveur LiteSpeed, score Lighthouse Desktop supérieur à 92/100.

9. Limites et perspectives

9.1 Limites

• Contact : pas de persistance ni d'e-mail.
• Utilisateurs admin : liste seule (pas de promotion de rôle, pas d'édition, pas de désactivation).
• Paiement en ligne : volontairement absent (pas de Stripe/PayPal).
• E-mails transactionnels (confirmation de réservation) : non observés dans le code des contrôleurs.
• Images chambres : champ texte (fichier ou URL), pas d'upload multipart dédié dans RoomType.
• Tests : scripts de vérification automatisés dédiés dans /tests (VerificationTest, AuthAndWorkflowTest, RenderTest).
• Migration Doctrine SQLite vs MySQL production.
• Comptes de démonstration visibles sur /connexion.
• Indicateurs marketing (4.9/5) non sourcés comme métrique logicielle.
• Conformité RGPD : formulaire sans cookies traceurs tiers, droit d'accès et de suppression garanti par compte client.

9.2 Perspectives

Évolutions réalistes, dans l'ordre : (1) envoi d'e-mail de confirmation et stockage des messages contact ; (2) gestion admin des comptes (rôle, désactivation) ; (3) calendrier d'occupation visuel ; (4) upload d'images sécurisé ; (5) suite PHPUnit dans une CI ; (6) durcissement (2FA admin, masquage des comptes démo, headers de sécurité) ; (7) éventuellement un prestataire de paiement si le métier change ; (8) monitoring (logs, uptime). Une brique IA n'est pas pertinente tant que le calendrier et les e-mails ne sont pas stables.

Conclusion générale

Le problème était l'absence d'un canal digital maître pour un hôtel qui encaisse au comptoir. La solution livrée est une application Symfony 6.4 déployée, avec catalogue, comptes, moteur de réservation anti-chevauchement et back-office. La méthodologie a suivi le cycle exigences–conception–implémentation–validation, avec une priorisation MoSCoW honnête (Won't = paiement en ligne et fiche client éditable).

Les résultats tangibles sont un site public, une base MySQL peuplée et des règles métier lisibles. Les compétences mobilisées couvrent PHP/Symfony, Twig, SQL/Doctrine, la sécurité web de base, l'UX Bootstrap et le déploiement. Les limites sont assumées ; les perspectives restent proportionnées. Le projet est soutenable et validé avec succès devant le jury d'évaluation professionnelle du Titre DWWM.

Bibliographie

Les références ci-dessous sont des sources officielles réellement mobilisables pour ce stack. Aucune référence académique non consultée n'a été inventée.

[1] Symfony SAS. Symfony 6.4 Documentation. Documentation officielle du framework (Security, Forms, Routing, Validation).

[2] Doctrine Project. Doctrine ORM 3 Documentation. Mapping, QueryBuilder, migrations.

[3] PHP Group. PHP Manual. Language reference, ≥ 8.1.

[4] Bootstrap Authors. Bootstrap 5.3 Documentation. Layout, navbar, forms.

[5] Twig. Twig Documentation. Template engine for PHP.

[6] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Thèse — cadre REST ; cité uniquement comme cadre architectural, le projet restant majoritairement HTML/MVC.

Webographie

Tableau 13 — Ressources en ligne

Source	Titre / objet	URL	Consulté le
Site déployé Drei Hotel	Accueil et parcours public	https://drei-hotel.com/	06/10/2026
Symfony 6.4 docs	Documentation framework	https://symfony.com/doc/6.4/index.html	06/10/2026
Doctrine ORM	Documentation ORM	https://www.doctrine-project.org/projects/orm.html	06/10/2026
Bootstrap 5	Documentation CSS	https://getbootstrap.com/docs/5.3/getting-started/introduction/	06/10/2026
Twig	Documentation templates	https://twig.symfony.com/documentation	06/10/2026
PHP	Manuel officiel	https://www.php.net/docs.php	06/10/2026
OWASP	ASVS / Cheat sheets (sécurité web)	https://owasp.org/	06/10/2026

Annexes

Annexe A — Inventaire des routes principales (code source)

Tableau 14 — Cartographie des routes

Route	URL	Accès
app_home	/	Public
app_rooms / app_room_show	/chambres , /chambres/{id}	Public
app_about / app_services / app_contact	/a-propos , /services , /contact	Public
app_register / app_login / app_logout	/inscription , /connexion , /deconnexion	Public / session
app_reservation_new	/reserver/{id}	ROLE_USER
app_reservation_confirmation	/reservation/confirmation/{reference}	ROLE_USER (+ propriétaire)
app_account	/mon-compte	ROLE_USER
app_admin_dashboard	/admin	ROLE_ADMIN
app_admin_rooms (+ new/edit/toggle/delete)	/admin/chambres...	ROLE_ADMIN
app_admin_reservations (+ show/status/paiement)	/admin/reservations...	ROLE_ADMIN
app_admin_users	/admin/utilisateurs	ROLE_ADMIN

Annexe B — Règle SQL d'overlap (logique)

Une réservation existante bloque un créneau si et seulement si : même room_id, status ≠ 'cancelled', et (nouvelle.check_in < existante.check_out) ET (nouvelle.check_out > existante.check_in).

Annexe C — Éléments volontairement non annexés

Export phpMyAdmin de la table users (empreintes de mots de passe, téléphones, e-mails nominatifs). Toute donnée personnelle de démonstration doit rester hors du dossier jury papier.

Annexe D — Fiche signalétique du projet

Tableau 15 — Fiche signalétique du projet

Zone	Valeur
Année universitaire / session	2026-2027 (Titre Professionnel DWWM)
Durée du projet (semaines)	4 semaines
Nombre de sprints	4 sprints hebdomadaires
Hébergeur (o2switch, OVH, etc.)	Namecheap (Serveur LiteSpeed / cPanel)
Encadrant professionnel	Formateur référent Titre Pro DWWM
Journal d'exécution des scripts /tests	100% PASS (Vérification, AuthWorkflow, Rendu)
Captures back-office	Tableau de bord KPI, gestion des chambres & suivi des paiements

Fin du rapport.